

UNIVERSITATEA TEHNICĂ A MOLDOVEI
Facultatea Calculatoare, Informatică și Microelectronică
Departamentul Ingineria Software și Automatică

LUCRARE DE LABORATOR NR. 4
The xv6 Shell from the Outside

Disciplina: Sisteme de Operare

A efectuat:
student gr. FAF-242
Daniel-Ioan Strunga

Au verificat:
asist. univ. Patricia Reițman
asist. univ. Marius Dumitrașcu

Chișinău — 2026

Cuprins

1 Part 1 & 2. Initial Checks & xv6 vs Bash **2**

2 Part 3. Who expands *? A program that prints its argv **2**
2.1 Implementare și testare în Linux Bash 2
2.2 Implementare pentru xv6 2

3 Part 4. Read the xv6 Shell (`sh.c`) **3**

4 Concluzie **4**

1 Part 1 & 2. Initial Checks & xv6 vs Bash

```
user@user-VPCEH1L0E:~/shtest$ echo hello > f ; cat < f
hello
user@user-VPCEH1L0E:~/shtest$ (echo a; echo b) | wc
      2       2       4
```

Observații: Mecanismele de conductă (*pipe*) și redirectare standard funcționează conform așteptărilor în shell-ul de bază al sistemului Linux.

2 Part 3. Who expands *? A program that prints its argv

2.1 Implementare și testare în Linux Bash

```c
#include <stdio.h>

int main(int argc, char *argv[]) {
    printf("argc = %d\n", argc);
    for(int i = 0; i < argc; i++)
        printf("argv[%d] = [%s]\n", i, argv[i]);
    return 0;
}
```

```
user@user-VPCEH1L0E:~/shtest$ gcc -o args args_linux.c
user@user-VPCEH1L0E:~/shtest$ ./args one two three
argc = 4
argv[0] = [./args]
argv[1] = [one]
argv[2] = [two]
argv[3] = [three]

user@user-VPCEH1L0E:~/shtest$ ./args *.c "x y"
argc = 3
argv[0] = [./args]
argv[1] = [args_linux.c]
argv[2] = [x y]
```

2.2 Implementare pentru xv6

Codul sursă a fost adaptat pentru xv6 utilizând apelurile de sistem definite în `user.h`.

```c
#include "kernel/types.h"
#include "user/user.h"

int
main(int argc, char *argv[])
{
  printf("argc = %d\n", argc);
  for(int i = 0; i < argc; i++)
    printf("argv[%d] = [%s]\n", i, argv[i]);
  exit(0);
}
```

Concluzie Part 3: Expansiunea caracterelor speciale (precum wildcard-ul `*`) este realizată în prealabil de către shell (ex. `bash`), nu de programul compilat în sine. Programul `args` primește lista deja evaluată prin intermediul argumentelor `argc` și `argv`.

3 Part 4. Read the xv6 Shell (`sh.c`)

```
user@user-VPCEH1L0E:~/xv6-riscv$ wc -l user/sh.c
499 user/sh.c

user@user-VPCEH1L0E:~/xv6-riscv$ grep -n "#define" user/sh.c
8:#define EXEC  1
9:#define REDIR 2
10:#define PIPE  3
11:#define LIST  4
12:#define BACK  5
14:#define MAXARGS 10

user@user-VPCEH1L0E:~/xv6-riscv$ grep -n "case " user/sh.c
75:  case EXEC:
83:  case REDIR:
93:  case LIST:
101:  case PIPE:
125:  case BACK:
284:  case 0:
286:  case '|':
287:  case '(':
288:  case ')':
289:  case ';':
290:  case '&':
291:  case '<':
294:  case '>':
391:    case '<':
394:    case '>':
397:    case '+': // >>
469:  case EXEC:
475:  case REDIR:
481:  case PIPE:
487:  case LIST:
493:  case BACK:

user@user-VPCEH1L0E:~/xv6-riscv$ grep -n "cd" user/sh.c | head -5
170:        fprintf(2, "cannot cd %s\n", cmd + 3);
```

Observații cod sursă shell (`sh.c`):

1. Fișierul sursă cuprinde 499 de linii de cod, ilustrând o implementare de referință, redusă, dar funcțională pentru un interpretor UNIX.

2. Arhitectura de procesare a comenzilor utilizează un Abstract Syntax Tree (AST) cu tipuri distincte pentru comenzi, definite între liniile 8 și 12: `EXEC`, `REDIR`, `PIPE`, `LIST`, `BACK`.

3. Cuvântul cheie `cd` este încorporat ca un *built-in* de bază (se regăsește în output la linia 170). Acesta trebuie interceptat direct de shell deoarece doar procesul părinte își poate schimba propriul director de lucru.

4 Concluzie

În urma realizării acestei lucrări de laborator, am dobândit o înțelegere practică asupra arhitecturii și modului de funcționare a unui interpretor de comenzi (shell) în mediul UNIX/xv6. Prin testarea comparativă a programului `args`, am demonstrat clar separarea responsabilităților între shell (care realizează expansiunea argumentelor și a wildcard-urilor) și programele executate (care le primesc deja prelucrate). Analizând structura codului sursă din `sh.c`, am observat modul eficient în care sistemul construiește și evaluează un Abstract Syntax Tree (AST) pentru a clasifica și executa comenzile simple, conductele (*pipes*) și redirectările, și am înțeles necesitatea implementării anumitor comenzi (precum `cd`) ca procese interne (*built-in*) ale shell-ului.